

Практическая работа. Модуль 4: графический интерфейс на Tkinter

Специальность 09.02.07 Информационные системы и программирование · демонстрационный экзамен, модуль 4

Язык: Python 3.12+ · Графика: tkinter (входит в поставку Python) · Структура проекта: два файла

СОДЕРЖАНИЕ

- 1. Минимум синтаксиса Python
- 2. Теория tkinter: окно, виджеты, отступы, сетка, диалоги, экраны
- 3. Практика: собираем приложение из двух файлов по шагам
- 4. Задания для самостоятельной работы
- 5. Типичные ошибки и как их читать
- 6. Контрольные вопросы

1. Минимум синтаксиса Python

Этот раздел — не полный курс, а тот минимум, без которого невозможно прочитать и написать код приложения из раздела 3. Каждый пункт сопровождается примером и тем, что программа напечатает.

1.1. Программа, вывод, комментарии

Программа на Python — это текст, который выполняется сверху вниз, строка за строкой. Команда `print` печатает значение на экран. Строка, начинающаяся с `#`, — комментарий: интерпретатор её игнорирует, комментарии пишут для людей. В коде самих приложений мы комментарии использовать не будем и объяснять код будем словами вокруг него, но понимать чужие комментарии необходимо.

```
# моя первая программа
print("Привет, мир")      # печать текста
print(2 + 3)              # печать результата вычисления
```

Вывод:

Привет, мир
5

1.2. Переменные и типы данных

Переменная — это имя, к которому привязано значение. Знак `=` означает «положить значение в переменную», а не «равно». Тип определяется автоматически по значению.

Тип	Название	Пример значения	Что хранит
-----	----------	-----------------	------------

int	целое число	42, -7	Количество, счётчик, идентификатор
float	дробное число	3.14, -0.5	Цена, сумма, процент
str	строка	"admin", 'текст'	Текст: логин, фамилия, название
bool	логический	True, False	Да/нет: заблокирован ли пользователь
None	пусто	None	«Значения пока нет»

```
login = "admin"
attempts = 3
price = 199.90
locked = False
current = None

print(type(login))
print(type(attempts))
```

Вывод:

```
<class 'str'>
<class 'int'>
```

ВАЖНО

Имена переменных пишутся строчными буквами, слова разделяются подчёркиванием: `user_login`, `failed_attempts`. Имя не может начинаться с цифры и не может содержать пробелов. Python различает регистр: `Login` и `login` — разные переменные.

1.3. Строки

Строку заключают в одинарные или двойные кавычки — разницы нет, но в одном проекте держатся одного стиля. Строки складывают (+), повторяют (*), берут символ по индексу и вырезают кусок.

```
first = "Иван"
last = "Петров"

full = first + " " + last
print(full)

print(len(full))
print(full[0])
print(full.upper())
print(full.lower())
print("  пробелы  ".strip())
```

Вывод:

```
Иван Петров
```

```
11
И
ИВАН ПЕТРОВ
иван петров
пробелы
```

f-строка — самый удобный способ подставить значение в текст. Перед открывающей кавычкой ставится буква `f`, а значения пишутся внутри фигурных скобок.

```
name = "admin"
count = 3
print(f"Пользователь {name} сделал {count} попытки")
```

Вывод:

Пользователь admin сделал 3 попытки

ВАЖНО

Складывать строку и число напрямую Python запрещает: пришлось бы писать `str(count)`. f-строка делает преобразование сама, поэтому в современном коде предпочитают её.

1.4. Числа и приведение типов

```
a = 7
b = 2
print(a + b)
print(a / b)
print(a // b)
print(a % b)
print(a ** 2)

s = "123"
n = int(s)
print(n + 1)
```

Вывод:

```
9
3.5
3
1
49
124
```

1.5. Ввод с клавиатуры в консольных программах

Функция `input()` останавливает программу и ждёт, пока пользователь введёт текст. Возвращает она **всегда строку**, даже если введены цифры, поэтому числа преобразуют.

```
name = input("Как вас зовут? ")
age = int(input("Сколько вам лет? "))
print(f"Здравствуйте, {name}. В прошлом году вам было {age - 1}.")
```

СВЯЗЬ С ГРАФИКОЙ

В графическом приложении `input()` не используется: текст вводят в поля ввода `Entry`, а читают методом `.get()`. Но понимание, что ввод приходит строкой, пригодится: `.get()` тоже возвращает строку.

1.6. Сравнения и логические операции

Знак	Значение	Пример при <code>a = 5</code>
<code>==</code>	равно	<code>a == 5</code> → <code>True</code>
<code>!=</code>	не равно	<code>a != 5</code> → <code>False</code>
<code>></code> , <code><</code>	больше, меньше	<code>a > 3</code> → <code>True</code>
<code>>=</code> , <code><=</code>	больше или равно, меньше или равно	<code>a >= 5</code> → <code>True</code>
<code>and</code>	и: <code>True</code> только если обе части <code>True</code>	<code>a > 3 and a < 10</code> → <code>True</code>
<code>or</code>	или: <code>True</code> если хотя бы одна часть <code>True</code>	<code>a > 100 or a < 10</code> → <code>True</code>
<code>not</code>	отрицание	<code>not (a == 5)</code> → <code>False</code>

Частая ошибка Одиночное `=` присваивает, двойное `==` сравнивает. Написать в условии `if login = "admin":` — ошибка синтаксиса. В условии всегда `==`.

1.7. Отступы и блоки: главное правило Python

В большинстве языков блоки кода ограничивают фигурными скобками. В Python блок определяется **отступом**. Все строки одного блока имеют одинаковый отступ; увеличение отступа начинает новый вложенный блок.

```
if True:
    print("эта строка внутри блока")
    print("и эта тоже")
print("а эта строка уже снаружи")
```

Вывод:

эта строка внутри блока
и эта тоже
а эта строка уже снаружи

ПРАВИЛА, КОТОРЫЕ НЕЛЬЗЯ НАРУШАТЬ

- Отступ делается пробелами, стандартно 4 пробела. Не смешивайте пробелы и табуляцию в одном файле.
- Перед началом блока ставится двоеточие: после `if`, `for`, `while`, `def`.
- Пустая строка блок не заканчивает; заканчивает его уменьшение отступа.
- Ошибка `IndentationError` почти всегда означает, что отступ съехал: либо пропущен, либо поставлен лишний.

1.8. Условие: `if`, `elif`, `else`

```
attempts = 3

if attempts >= 3:
    print("Пользователь заблокирован")
elif attempts == 2:
    print("Осталась одна попытка")
else:
    print("Можно продолжать")
```

Вывод:

Пользователь заблокирован

Порядок проверки сверху вниз: сработает первая ветвь, условие которой истинно, остальные пропускаются. Ветвей `elif` может быть несколько, ветвь `else` необязательна.

Проверка «пусто ли» пишется через `not`: пустая строка, пустой список и ноль считаются ложными значениями.

```
login = " ".strip()

if not login:
    print("Логин не введен")
else:
    print("Логин введен")
```

Вывод:

Логин не введен

1.9. Списки

Список хранит несколько значений по порядку в одной переменной. Элементы нумеруются с нуля.

```
roles = ["Администратор", "Пользователь", "Гость"]

print(roles[0])
print(len(roles))
roles.append("Аудитор")
print(roles)

for r in roles:
    print("-", r)
```

Вывод:

```
Администратор
3
['Администратор', 'Пользователь', 'Гость', 'Аудитор']
- Администратор
- Пользователь
- Гость
- Аудитор
```

1.10. Словари

Словарь хранит пары «ключ — значение». Это основной способ описать одну запись: одного пользователя, один товар. К значению обращаются по ключу в квадратных скобках.

```
user = {"login": "admin", "role": "Администратор", "locked": False}

print(user["login"])
print(user["locked"])

user["locked"] = True
print(user["locked"])

for key in user:
    print(key, "=", user[key])
```

Вывод:

```
admin
False
True
login = admin
role = Администратор
locked = True
```

Список словарей — простейшая «таблица»: несколько записей одинаковой структуры. Именно так в нашей практике устроена заглушка базы данных.

```
USERS = [  
    {"login": "admin", "role": "Администратор"},  
    {"login": "ivanov", "role": "Пользователь"},  
]  
  
for u in USERS:  
    print(u["login"], "-", u["role"])
```

Вывод:

admin – Администратор
ivanov – Пользователь

1.11. Циклы for и while

```
for i in range(1, 4):  
    print("попытка", i)  
  
n = 3  
while n > 0:  
    print("осталось", n)  
    n = n - 1  
  
for i in range(1, 10):  
    if i == 4:  
        break  
    print(i)
```

Вывод:

попытка 1
попытка 2
попытка 3
осталось 3
осталось 2
осталось 1
1
2
3

1.12. Функции

Функция — именованный блок кода, который можно вызывать много раз. Объявляется словом `def`. В скобках перечисляются параметры — значения, которые функция получает снаружи. Команда `return` возвращает результат и завершает функцию.

```
def greet(name):  
    return "Привет, " + name  
  
message = greet("admin")  
print(message)  
  
def is_adult(age):  
    if age >= 18:  
        return True  
    return False  
  
print(is_adult(20))  
print(is_adult(15))
```

Вывод:

```
Привет, admin  
True  
False
```

ВАЖНО

Если в функции нет `return`, она вернёт `None`. Функция, которая только печатает, ничего не возвращает. Различайте: `print` показывает значение человеку, `return` отдаёт значение программе.

1.13. Область видимости и слово `global`

Переменная, созданная внутри функции, существует только внутри неё и снаружи не видна. Переменная, созданная на верхнем уровне файла, видна всем как **для чтения**. Но если функция хочет такую переменную **перезаписать**, она должна объявить её словом `global`.

```
CURRENT_USER = None  
  
def login_ok(name):  
    global CURRENT_USER  
    CURRENT_USER = name  
  
login_ok("admin")  
print(CURRENT_USER)
```

Вывод:

```
admin
```

ГДЕ ВСТРЕТИТСЯ В ПРАКТИКЕ

В приложении есть переменная `CURRENT_USER` — кто сейчас вошёл. Функция проверки логина её перезаписывает, поэтому в ней стоит строка `global CURRENT_USER`. Без неё после входа переменная осталась бы пустой и следующий экран не узнал бы, кто вошёл.

1.14. Подключение модулей: `import`

Дополнительные возможности лежат в модулях и подключаются командой `import` в начале файла.

```
import tkinter
from tkinter import messagebox

root = tkinter.Tk()
messagebox.showinfo("Тест")
```

В нашей практике начало файла всегда выглядит так:

```
import tkinter as tk
from tkinter import ttk, messagebox
```

Первая строка подключает модуль целиком под коротким именем `tk`, чтобы не писать `tkinter` каждый раз. Вторая достает из модуля два конкретных объекта: `ttk` (улучшенные виджеты, среди них таблица и выпадающий список) и `messagebox` (диалоговые окна). Тем же приёмом `from ... import ...` мы позже подключим собственный файл с моделью пользователя.

1.15. Обработка ошибок: `try / except`

Если в коде произойдёт ошибка, программа аварийно завершится и покажет `traceback`. Блок `try / except` перехватывает ошибку и позволяет продолжить работу аккуратно. Для графического приложения это обязательный приём: программа не должна падать ни при каком вводе пользователя.

```
text = "abc"
try:
    number = int(text)
    print("число:", number)
except ValueError:
    print("Это не число, попробуйте ещё раз")
print("программа продолжает работать")
```

Вывод:

Это не число, попробуйте ещё раз
программа продолжает работать

1.16. Анонимные функции: `lambda`

`lambda` — короткая запись функции из одного выражения, без имени и без `return`. Результат выражения и есть возвращаемое значение. `Lambda` понадобится, чтобы передать кнопке действие с аргументами.

```
double = lambda x: x * 2
print(double(5))

greet = lambda name: "Привет, " + name
print(greet("admin"))
```

Вывод:

10

Привет, admin

Самопроверка по разделу 1 Не заглядывая выше, ответьте:

- Что напечатает `print(len("abcd"))` и `print("abcd"[0])`?
- Чем `=` отличается от `==`?
- Почему `int(input())` оборачивают в `int`?
- Что вернёт функция без `return`?
- Зачем в функции, меняющей `CURRENT_USER`, нужна строка `global`?

2. Теория tkinter: окно, виджеты, отступы, сетка, диалоги, экраны

2.1. Что такое tkinter и логика графического приложения

tkinter — стандартная библиотека графического интерфейса Python. Она устанавливается вместе с интерпретатором, ничего докупать не нужно, и она одинаково работает на Windows и Linux. Для модуля 4 этого достаточно: интерфейс экзаменационного приложения — это окна, надписи, поля ввода, кнопки и таблицы, и всё это в tkinter есть.

Логика любого графического приложения одна и та же:

1. Создаётся главное окно.
2. В окне размещаются виджеты — видимые элементы управления.
3. Запускается цикл ожидания событий: программа не завершается, а ждёт, пока пользователь нажмёт кнопку, введёт текст или закроет окно.
4. На каждое действие пользователя заранее «подписывается» функция-обработчик. Нажал кнопку — вызвалась её функция.

2.2. Главное окно и цикл событий

```
import tkinter as tk

root = tk.Tk()
root.title("Учебное приложение")
root.geometry("640x480")
root.configure(bg="#f9f1e5")

root.mainloop()
```

Команда	Назначение
<code>tk.Tk()</code>	Создать главное окно. Вызывается один раз.
<code>root.title(...)</code>	Текст в заголовке окна.
<code>root.geometry("ШxB")</code>	Начальный размер окна. Можно добавить координаты: <code>"640x480+100+50"</code> .
<code>root.configure(bg=...)</code>	Изменить свойство окна, чаще всего цвет фона.

<code>root.mainloop()</code>	Запустить цикл событий. Ставится последней строкой файла.
------------------------------	---

ПОЧЕМУ ОКНО «МИГАЕТ И ЗАКРЫВАЕТСЯ»

Строки программы выполняются сверху вниз. Если после создания окна не вызвать `mainloop()`, программа дойдёт до конца файла и завершится, закрыв окно. `mainloop()` «держит» программу живой: он крутится в бесконечном цикле и раздаёт события виджетам, пока окно не закроют.

2.3. Виджеты и родитель

Виджет — любой видимый элемент: надпись, поле, кнопка, рамка, таблица. У каждого виджета есть **родитель** — контейнер, внутри которого он лежит. Родитель передаётся первым аргументом при создании.

```
label = tk.Label(root, text="Привет")
label.pack()
```

После создания виджет нужно разместить одним из менеджеров размещения: `pack()`, `grid()` или `place()`. Без этого виджет существует, но не виден на экране.

2.4. Обзор виджетов, которые понадобятся

Виджет	Зачем	Как создать
<code>tk.Label</code>	Надпись, заголовок, пояснение	<code>tk.Label(root, text="Логин")</code>
<code>tk.Entry</code>	Однострочное поле ввода	<code>tk.Entry(root)</code> ; пароль: <code>show="*"</code>
<code>tk.Button</code>	Кнопка, вызывает функцию	<code>tk.Button(root, text="Войти", command=f)</code>
<code>tk.Frame</code>	Невидимый контейнер-рамка для группировки	<code>tk.Frame(root)</code>
<code>tk.Checkbutton</code>	Флажок «да/нет»	<code>tk.Checkbutton(root, text="...", variable=v)</code>
<code>ttk.Combobox</code>	Выпадающий список	<code>ttk.Combobox(root, values=[...])</code>
<code>ttk.Treeview</code>	Таблица со строками и колонками	<code>ttk.Treeview(root, columns=(...), show="headings")</code>
<code>messagebox</code>	Всплывающее диалоговое окно-сообщение	<code>messagebox.showinfo("Заголовок", "Текст")</code>
<code>tk.Canvas</code>	Холст для рисования; понадобится позже для капчи	<code>tk.Canvas(root, width=180, height=55)</code>

2.5. Параметры виджетов, которые встречаются в практике

Параметр	Назначение	Пример
text	Надпись на виджете	text="Войти"
font	Шрифт: семейство, размер, начертание	font=("Arial", 12, "bold")
bg	Цвет фона виджета	bg="#1e5fd0"
fg	Цвет текста виджета	fg="white"
activebackground	Цвет фона кнопки в момент нажатия	activebackground="#174ea6"
activeforeground	Цвет текста кнопки в момент нажатия	activeforeground="white"
relief	Объём рамки: flat, raised, sunken, groove	relief="flat"
cursor	Вид курсора над виджетом	cursor="hand2" — рука над кнопкой
show	У Entry: каким символом заменять вводимое	show="*" для пароля
anchor	Куда прижать текст внутри виджета	anchor="w" — к левому краю
justify	Выравнивание строк многострочного текста	justify="left"
wraplength	У Label: ширина, после которой переносить строки	wraplength=560
command	У Button: функция, вызываемая по нажатию	command=open_admin

2.6. Цвета

Цвет задаётся строкой, именем или шестнадцатеричным кодом.

Способ	Примеры
Имя	"white", "black", "red", "gray", "lightblue"
Код #RRGGBB	"#f9f1e5" пастельный персиковый фон, "#eef4e4" пастельный салатный, "#1e5fd0" синий, "#0f7b46" зелёный, "#b42318" красный

В коде #RRGGBB пары RR, GG, BB — интенсивность красного, зелёного и синего от 00 до FF. Чем больше значение пары, тем ярче соответствующая составляющая.

ГДЕ СТАВИТЬ ЦВЕТ

Цвет фона окна задаётся один раз: `root.configure(bg=...)`. Цвет фона конкретного виджета — параметром `bg` при его создании. Если у виджета не задать `bg`, он возьмёт системный цвет, который может не совпасть с цветом окна, и виджет будет выглядеть «заплаткой». Поэтому мы задаём `bg` явно почти каждому виджету и храним общий цвет в константе `BG`.

Ограничение, о котором надо знать Изменить цвет уже созданного виджета можно методом `configure(bg=...)`. Но если перекрасить окно, ранее созданные виджеты с собственным `bg` не перекрасятся сами. Поэтому общий фон задают до создания виджетов, а локальные перекраски делают точно, обращаясь к конкретному виджету.

ФОН И ПОЛЯ ВВОДА

Для фона окна используйте светлые пастельные оттенки, тёмный фон не применяйте. Поля ввода окрашиваются явно: белый фон `FIELD_BG`, тёмный текст `FIELD_FG`, цвет курсора `insertbackground` и видимая рамка `relief="solid"`. Такое поле читается на любом фоне.

Все цвета и шрифты — в константах

Все цвета и шрифты приложения объявляются один раз в начале файла именованными константами, а дальше весь код ссылается только на имена. Перекраска всего интерфейса сводится к правке нескольких строк в одном месте.

```
BG = "#f9f1e5"
FIELD_BG = "#ffffff"
FIELD_FG = "#22262b"
PRIMARY_BG = "#1e5fd0"
PRIMARY_FG = "#ffffff"
SECOND_BG = "#e5e7eb"
SECOND_FG = "#111827"

FONT_TITLE = ("Arial", 16, "bold")
FONT_LABEL = ("Arial", 11, "bold")
FONT_BUTTON = ("Arial", 12, "bold")
```

Смысл имён прямой: `BG` — общий фон окна и контейнера, `FIELD_BG` и `FIELD_FG` — фон и текст полей ввода, `PRIMARY` — стиль главной кнопки экрана, `SECOND` — стиль всех остальных кнопок, три `FONT` — заголовок, подпись и кнопка. Больше ни один hex-код и ни один кортеж шрифта в экранах не встречается: оформление вынесено в функции-строители `make_label`, `make_entry`, `make_button`, описанные в практике.

Установка и проверка tkinter: Windows и Linux

`tkinter` входит в состав Python и не устанавливается через `pip`, однако в некоторых системах он поставляется отдельным пакетом. Наличие проверяется одной командой в терминале:

```
python -c "import tkinter; print('ok')"
```

Если команда напечатала `Ok`, библиотека готова к работе и дальше можно не читать. Если появилась ошибка `ModuleNotFoundError` или обрыв на строке `import _tkinter`, выполните шаг своей операционной системы и повторите проверку.

Шаг 1 Windows. `tkinter` входит в установщик Python. Запустите установщик повторно, выберите `Modify` и убедитесь, что отмечен компонент «`tcl/tk and Idle`». Либо переустановите Python, на первом экране оставив галочку «`Add python.exe to PATH`». Повторите проверку.

Шаг 2 Linux, Debian / Ubuntu и производные. Выполните `sudo apt update`, затем `sudo apt install python3-tk`. Повторите проверку.

Шаг 3 Linux, Fedora / RHEL. Выполните `sudo dnf install python3-tkinter`. Повторите проверку.

Шаг 4 Linux, ALT. Выполните `apt-get install python3-module-tkinter` с правами администратора. Повторите проверку.

Шаг 5 Финальная проверка запуска. Запустите `app.py`. Должно открыться окно со светлым пастельным фоном, белыми полями ввода с рамкой и двумя кнопками на экране входа.

2.7. Отступы: внешний (margin) и внутренний (padding)

В вёрстке различают два вида пустого пространства вокруг и внутри элемента, и в tkinter им соответствуют разные параметры. Понимать разницу между ними — значит уметь управлять «воздухом» интерфейса, а не получать слипшиеся или разлетевшиеся виджеты.

Внешний отступ: расстояние от виджета до соседей

Внешний отступ (в веб-терминологии `margin`, «поле») — это пустое место **снаружи** виджета, между его рамкой и соседними виджетами или стенкой контейнера. Он не меняет размер самого виджета, он лишь раздвигает его с окружением. В менеджерах `pack` и `grid` внешний отступ задаётся параметрами `padx` (по горизонтали) и `pady` (по вертикали).

Внутренний отступ: расстояние внутри виджета

Внутренний отступ (в веб-терминологии `padding`, «набивка») — это пустое место **внутри** виджета, между его рамкой и его содержимым, то есть текстом надписи или кнопки. Он увеличивает сам виджет, делая кнопку толще или шире, не меняя при этом размер шрифта. В `pack` и `grid` внутренний отступ задаётся параметрами `ipadx` и `ipady` (буква *i* означает *internal*, внутренний).

Параметр	Тип отступа	Что меняет	Когда применять
<code>padx</code>	внешний, горизонталь	Зазор слева и справа от виджета	Развести кнопки в ряду, отступить от края окна
<code>pady</code>	внешний, вертикаль	Зазор сверху и снизу от виджета	Разделить поля формы по вертикали
<code>ipadx</code>	внутренний, горизонталь	Ширина самого виджета	Сделать кнопку шире без длинного текста
<code>ipady</code>	внутренний, вертикаль	Высота самого виджета	Сделать кнопку и поле «толще», удобнее для нажатия

Наглядный пример: две кнопки с одинаковым текстом. Первая выглядит тонкой полоской, вторая — полноценной крупной кнопкой, хотя шрифт одинаковый.

```
tk.Button(container, text="Войти").pack(fill="x")
tk.Button(container, text="Войти").pack(fill="x", ipady=6)
```

Вторая строка добавляет по шесть пикселей внутреннего отступа сверху и снизу, и кнопка становится выше ровно на двенадцать пикселей. Именно так в практике сделана кнопка «Войти»: крупная и удобная, без раздувания шрифта.

Отступы только с нужных сторон

Оба параметра принимают не только число, но и пару чисел в скобках: первое — для левой или верхней стороны, второе — для правой или нижней.

```
label.pack(fill="x", pady=(0, 10))
button.pack(side="left", padx=(0, 6))
```

Первая строка даёт ноль сверху и десять пикселей снизу: заголовок прижат к тому, что выше, но отделён от того, что ниже. Вторая даёт ноль слева и шесть справа: кнопки в ряду стоят вплотную друг к другу с зазором шесть пикселей, но не прилипают. Такой точечный подход аккуратнее, чем одинаковый отступ со всех сторон, потому что не удваивает зазор между соседями.

ГДЕ ВНУТРЕННИЙ ОТСТУП ОСОБЕННО ПОЛЕЗЕН

- Кнопки: `pady` делает их заметными и удобными для попадания курсором, не увеличивая шрифт.
- Поля ввода: небольшой `pady` даёт тексту «воздух» внутри поля, ввод читается легче.
- Кнопки в одном ряду: `padx` выравнивает их по ширине, когда надписи разной длины.

2.8. Менеджеры размещения и сетка grid

Менеджер размещения решает, где внутри контейнера встанет виджет и какого он будет размера. В tkinter их три, и в одном контейнере можно использовать только один — смешивание даёт ошибку.

Менеджер	Как раскладывает	Когда выбирать
pack	Складывает виджеты стопкой или в ряд вдоль выбранной стороны	Простые вертикальные формы, ряд кнопок. Самый простой, с него начинаем
grid	Раскладывает виджеты по невидимой таблице строк и колонок	Формы «надпись слева, поле справа», любые табличные раскладки
place	Ставит виджет в точные координаты в пикселях	Почти никогда в учебных работах: раскладка не адаптируется к размеру окна

Что такое сетка grid

Сетка — это воображаемая таблица, натянутая поверх контейнера. У неё есть строки и колонки, нумеруемые с нуля. Виджет помещается в ячейку указанием пары «строка, колонка». Ячейка может быть не одной: виджет способен растянуться на несколько колонок или строк. Размеры строк и колонок tkinter подбирает сам по самому крупному содержимому, поэтому сетку не нужно размечать вручную.

```
tk.Label(container, text="Логин").grid(row=0, column=0, sticky="w", pady=5)
tk.Entry(container).grid(row=0, column=1, sticky="ew", pady=5)

tk.Label(container, text="Пароль").grid(row=1, column=0, sticky="w", pady=5)
tk.Entry(container, show="*").grid(row=1, column=1, sticky="ew", pady=5)

tk.Button(container, text="Войти").grid(row=2, column=0, colspan=2, sticky="ew", pady=10)

container.columnconfigure(1, weight=1)
```

Разбор примера. Надпись «Логин» стоит в строке 0 колонке 0 и прижата к левому краю ячейки (`sticky="w"`). Поле ввода стоит в той же строке, но в колонке 1, и растянуто по ширине ячейки (`sticky="ew"` — на восток и запад одновременно). Кнопка занимает строку 2 и растянулась на обе колонки (`colspan=2`), поэтому она шире любого поля. Последняя строка

сообщает сетке, что при растягивании окна дополнительное место должна получать колонка 1 (колонка с полями), а не колонка 0 с надписями: без этого при расширении окна поля остались бы узкими, а справа появилась бы пустота.

Аргумент grid	Действие
row, column	Номер строки и колонки ячейки, с нуля
rowspan, colspan	На сколько строк или колонок растянуть виджет
sticky	К каким краям ячейки прижать виджет: "w" лево, "e" право, "n" верх, "s" низ; комбинации "ew", "nw" и так далее
padx, pady	Внешние отступы вокруг виджета внутри ячейки
ipadx, ipady	Внутренние отступы, увеличивающие сам виджет
columnconfigure(i, weight=1)	Разрешить колонке i забирать свободное место при растягивании окна
rowconfigure(i, weight=1)	То же для строки

Какой менеджер проще всего

Для старта проще всего `pack`: он не требует думать о строках и колонках, виджеты сами встанут друг под другом, а ряд кнопок делается через `side="left"`. К `grid` переходят, когда появляется форма с парами «надпись — поле» или нужно выровнять несколько колонок: там `pack` потребовал бы вложенных рамок, а `grid` даёт выравнивание бесплатно. Наша практика сознательно построена на `pack`, чтобы не смешивать две новые темы сразу, но задание на перевод экрана входа на `grid` в разделе 4 есть, и к этому моменту сетка уже знакома по этому параграфу.

ЗапретВнутри одного контейнера нельзя смешивать менеджеры. Если часть виджетов уложена `pack`, а часть `grid`, `tkinter` остановит программу с ошибкой `TclError` о конфликте менеджеров. Один контейнер — один менеджер. Если нужны разные раскладки рядом, создайте вложенные `Frame` и в каждом используйте свой менеджер.

2.9. Чтение и запись значений

```
entry = tk.Entry(root)
entry.pack()

text = entry.get()
entry.delete(0, "end")
entry.insert(0, "admin")
```

У выпадающего списка `ttk.Combobox` значение читают тем же `.get()`; задать выбранное программно можно `.current(номер)` или `.set("текст")`. Параметр `state="readonly"` запрещает вводить своё значение, оставляя только выбор из списка.

ВАЖНО

`.get()` возвращает строку, даже если пользователь ввёл цифры. Прежде чем считать количество или сумму, преобразуйте: `int(entry.get())`, и оберните в `try / except ValueError`, чтобы ввод букв не уронил программу.

2.10. Кнопки и command: самая частая ошибка новичков

Параметр `command` принимает **саму функцию**, без скобок. Кнопка вызовет её позже, когда пользователь нажмёт.

```
def open_admin():
    print("открыли админку")

btn = tk.Button(root, text="Админка", command=open_admin)
bad = tk.Button(root, text="Админка", command=open_admin())
```

Почему со скобками ошибка Запись `open_admin()` немедленно вызывает функцию ещё во время построения окна. Кнопке же достаётся то, что функция вернула (обычно `None`), и нажатие не делает ничего. Запомните: **скобки — вызвать сейчас, без скобок — отдать кнопке, чтобы вызвала потом**.

Если в функцию нужно передать аргументы, например значения из полей ввода, используют `lambda`. `lambda` создаёт функцию без аргументов для кнопки, внутри которой вызывается нужная функция с нужными аргументами.

```
btn = tk.Button(root, text="Войти",
                command=lambda: try_login(entry_login.get(), entry_pass.get()))
```

Здесь `lambda:` означает «функция без параметров, тело которой — выражение после двоеточия». Кнопка вызовет её по нажатию, и лишь тогда выполнятся `.get()` и `try_login` с актуальными значениями полей. Если написать `command=try_login(entry_login.get(), ...)` без `lambda`, всё вычислится сразу при построении экрана — та же ошибка, что со скобками.

2.11. Диалоговые окна messagebox

Диалоговое окно — это отдельное небольшое окно, которое программа открывает поверх главного, чтобы сообщить пользователю результат или задать вопрос. Оно является **модальным**: пока диалог открыт, всё остальное приложение недоступно, курсор и клавиатура работают только с диалогом, а выполнение кода остановилось ровно на той строке, которая диалог вызвала. Как только пользователь закрывает диалог кнопкой, выполнение продолжается со следующей строки. Именно поэтому диалог удобно использовать как точку, в которой программа «ждёт решения человека».

Модальность даёт ещё одно важное свойство: пользователь физически не может пропустить сообщение. В консольной программе вывод `print` легко не заметить, а у графического приложения консоль вообще может быть скрыта. Диалог гарантирует, что человек увидит и ошибку «логин занят», и подтверждение «пользователь добавлен», и вопрос «выйти?».

```
from tkinter import messagebox

messagebox.showinfo("Готово", "Пользователь добавлен")
messagebox.showwarning("Внимание", "Заполните все поля")
messagebox.showerror("Ошибка", "Неверный логин или пароль")

if messagebox.askyesno("Выход", "Выйти из программы?"):
    root.destroy()
```

Функция	Зачем	Что возвращает
<code>showinfo</code>	Сообщить об успешном действии	Ничего полезного; вызывается ради показа
<code>showwarning</code>	Предупредить о неверном, но не фатальном вводе	То же

showerror	Сообщить об отказе: неверный пароль, дубликат, блокировка	То же
askyesno	Задать вопрос с выбором Да/Нет, обычно перед необратимым действием	True при «Да», False при «Нет»
askyesnocancel	Три варианта: Да/Нет/Отмена	True, False или None

Первый аргумент всегда заголовок диалога, второй — текст сообщения. Хорошая практика: в заголовке писать тип ситуации («Ошибка», «Внимание», «Готово»), а в тексте — конкретику, что именно произошло и что делать дальше.

2.12. Таблица ttk.Treeview

Treeview показывает табличные данные: строки и колонки с заголовками. Это основной способ вывести список пользователей в админ-панели.

```
tree = ttk.Treeview(container, columns=("login", "role", "locked"),
                    show="headings", height=8)

tree.heading("login", text="Логин")
tree.heading("role", text="Роль")
tree.heading("locked", text="Заблокирован")

tree.column("login", width=180, anchor="w")
tree.column("role", width=160, anchor="w")
tree.column("locked", width=120, anchor="center")

tree.insert("", "end", values=("admin", "Администратор", "нет"))
tree.pack(fill="both", expand=True)
```

Команда	Назначение
columns(...)	Имена колонок (внутренние, не видны пользователю)
show="headings"	Скрыть служебную первую колонку с деревом; для плоской таблицы обязательно
height=N	Сколько строк показывать без прокрутки
tree.heading(имя, text=...)	Видимый заголовок колонки
tree.column(имя, width=..., anchor=...)	Ширина колонки и выравнивание текста в ней
tree.insert("", "end", values=(...))	Добавить строку; values — значения по колонкам по порядку
tree.selection()	Список идентификаторов выделенных строк; пуст, если ничего не выделено
tree.item(id) ["values"]	Значения строки по идентификатору

Типовой приём «взять выделенную строку»:

```
selected = tree.selection()
if not selected:
    messagebox.showwarning("Внимание", "Сначала выберите строку.")
else:
    login = tree.item(selected[0])["values"][0]
```

2.13. Файлы проекта, модель и экраны

Один файл или несколько

Отвечая на частый вопрос: да, небольшое приложение на tkinter действительно можно написать в одном файле, где каждая задача оформлена отдельной функцией. Так делают прототипы и примеры. Проблема начинается, когда в одном файле сосуществуют две принципиально разные вещи: **данные и правила работы с ними** и **виджеты с раскладкой**. Файл разрастается, в нём невозможно найти нужное, а любое изменение данных рискует задеть интерфейс.

Поэтому проект делится на файлы по ответственности. В нашей практике файлов два:

Файл	Что внутри	Чего внутри быть не должно
<code>models.py</code>	Модель: список-заглушка пользователей и функции поиска, добавления, разблокировки	Ни одного виджета, ни одного импорта tkinter
<code>app.py</code>	Окно, контейнер, все экраны и переходы, диалоги	Прямых переборов списка пользователей и правил «кто кого блокирует»

Зачем модель вынесена отдельно

- **Замена данных без переделки интерфейса.** Сейчас модель хранит список в памяти. На следующем шаге обучения список заменится на базу sqlite3: изменится только содержимое функций в `models.py`, а все экраны в `app.py` останутся нетронутыми, потому что вызывают те же имена функций.
- **Читаемость.** Экран входа отвечает только за поля и кнопку; правило «пароль не подошёл — вернуть None» живёт в модели и написано один раз.
- **Проверяемость.** Логiku модели можно протестировать, не запуская окно: достаточно импортировать `models` и позвать функции.

Как файлы подключаются друг к другу

Файлы лежат в одной папке проекта. В начале `app.py` пишется строка импорта, и Python находит `models.py` по имени без расширения:

```
from models import all_users, find_user, login_exists, add_user, unlock_user
```

После этой строки все перечисленные функции доступны в `app.py` так, будто объявлены прямо в нём. Второй вариант записи — `import models`, и тогда каждое обращение пишется с префиксом: `models.find_user(...)`. Первый вариант короче и потому удобнее; второй полезен, когда имён много и хочется явно видеть, откуда пришло каждое.

Зачем нужны разные экраны и как они устроены

Экран — это набор виджетов, который пользователь видит в один момент времени: вход, админ-панель, форма добавления. В tkinter нет готовых «страниц», поэтому каждый экран оформляют **функцией**. Функция экрана делает две вещи: очищает

контейнер от виджетов предыдущего экрана и строит в контейнере свои виджеты. Переход между экранами — это просто вызов функции другого экрана, обычно из обработчика кнопки.

Почему все функции экранов живут в одном `app.py`, а не разложены по файлам на экран: экраны вызывают друг друга по кругу (вход открывает админку, админка открывает форму добавления, форма возвращает в админку, админка возвращает на вход). Если бы каждый экран был отдельным модулем, импортирующим следующий, получились бы круговые импорты и трудные для понимания ошибки загрузки. Один модуль для всех экранов плюс один модуль для модели — правильный баланс для учебного проекта. Дробить дальше начинают не по экранам, а по группам виджетов, когда файл экранов становится действительно большим.

Схема переходов нашего приложения

ПЕРЕХОДЫ

Вход → (успешная проверка) Панель администратора.

Вход → (кнопка «О программе») О программе → (кнопка «Назад») Вход.

Панель администратора → (кнопка «Добавить») Новый пользователь → (Сохранить или Отмена) Панель администратора.

Панель администратора → (кнопка «Оформление») Оформление → (кнопка «Назад») Панель администратора.

Панель администратора → (кнопка «Выйти») Вход.

Обратите внимание: возврат на предыдущий экран — это не «назад» в смысле браузера, а обычный вызов функции нужного экрана. Экран при этом пересоздаётся с нуля, и таблица показывает свежие данные. Это же объясняет, почему после изменения данных мы вызываем функцию экрана повторно: перерисовка в `tkinter` делается пересозданием виджетов.

Кнопки возврата на экранах

У каждого экрана, кроме стартового, есть кнопка, возвращающая на шаг назад или к точке входа. Стартовый экран входа кнопкой возврата не обладает, потому что возвращаться некуда; вместо него на нём стоит переход «О программе».

Экран	Кнопка возврата	Куда приводит
Вход	нет, это стартовый экран	Дополнительно есть переход «О программе»
Панель администратора	Выйти	Вход
Рабочее место пользователя	Выйти	Вход
Новый пользователь	Отмена	Панель администратора
О программе	Назад	Вход
Оформление	Назад	Панель администратора

3. Практика: собираем приложение из двух файлов по шагам

Собираем приложение с шестью экранами и интерактивной капчей-пазлом. Данные пока хранятся в списке-заглушке в `models.py`; подключение базы будет позже, и структура проекта к этому готова. Код приложений приводится без комментариев: каждый кусок разбирается словами сразу под ним. При этом техническое задание модуля 4 требует пояснять комментариями неочевидные фрагменты и давать осмысленные имена, поэтому в итоговой экзаменационной работе комментарии добавить нужно; здесь они заменены разбором в тексте, чтобы не мешать чтению.

Требование модуля 4	Где отрабатывается
Форма авторизации	Шаг 8, экран входа
Интерфейс администратора: таблица пользователей и добавление	Шаги 9 и 11
Контроль уникальности логина	Шаг 11, функция <code>save_user</code>
Переходы между экранами и кнопки возврата	Шаги 8–13 и подраздел 2.13
Интерактивная капча: сборка изображения из фрагментов	Шаг 7
Счётчик неудачных попыток и блокировка после трёх неудач подряд	Шаг 3 (модель) и шаг 8 (вход)
Изменение пользователя, включая снятие блокировки	Задание 8; снятие блокировки также в шаге 9
Обработка исключений, отсутствие аварийных завершений	Раздел 5 и блок «Проверьте себя»
Соответствие интерфейса макету из технического задания	На экзамене сверяться с макетом Приложения 1

Шаг 1 Создать папку проекта и два файла. Создайте папку с именем без пробелов, например `module4_app`, откройте её в редакторе и создайте внутри два пустых файла: `models.py` и `app.py`. Первый будет моделью, второй интерфейсом. Проверьте, что в `models.py` на протяжении всей работы не появится ни одной строки с `tkinter`: если появилась, значит интерфейс начал протекать в модель и файлы пора распутывать.

Шаг 2 `models.py`: список-заглушка пользователей. Это и есть модель пользователя. Каждая запись — словарь с четырьмя полями: логин, пароль, роль и признак блокировки. Признак блокировки хранит логическое значение, а не строку, потому что по нему дальше будут приниматься решения.

```
USERS = [
    {"login": "admin", "password": "admin", "role": "Администратор",
     "locked": False, "attempts": 0},
    {"login": "ivanov", "password": "1234", "role": "Пользователь",
     "locked": True, "attempts": 3},
    {"login": "petrov", "password": "qwerty", "role": "Пользователь",
     "locked": False, "attempts": 0},
]
```

Здесь создаётся сама модель: три готовые записи, на которых будем проверять вход, блокировку и добавление. Запись `ivanov` специально заблокирована, чтобы сразу видеть работу отказа по флагу `locked`.

Шаг 3 models.py: функции работы с моделью. Ниже пять функций. Первая отдаёт весь список, чтобы интерфейс мог построить таблицу, не зная устройства хранения. Вторая ищет пару логин-пароль и возвращает найденный словарь либо `None`, если пары нет; возврат `None` — это соглашение «не нашли», и вызывающий код проверяет его через `is None`. Третья отвечает на вопрос «занят ли логин» до добавления. Четвёртая дописывает новую запись в список. Пятая снимает блокировку по логину и возвращает `True` или `False`, чтобы вызывающий код знал, была ли такая запись вообще.

```
def all_users():
    return USERS

def find_user(login, password):
    for u in USERS:
        if u["login"] == login and u["password"] == password:
            return u
    return None

def login_exists(login):
    for u in USERS:
        if u["login"] == login:
            return True
    return False

def add_user(login, password, role):
    USERS.append({"login": login, "password": password,
                  "role": role, "locked": False, "attempts": 0})

def register_fail(login):
    for u in USERS:
        if u["login"] == login:
            u["attempts"] = u["attempts"] + 1
            if u["attempts"] >= 3:
                u["locked"] = True
                return True
            return False
    return False

def reset_attempts(login):
    for u in USERS:
        if u["login"] == login:
            u["attempts"] = 0
            return True
    return False

def unlock_user(login):
    for u in USERS:
        if u["login"] == login:
            u["locked"] = False
            u["attempts"] = 0
            return True
    return False
```

Обратите внимание: ни одна функция ничего не печатает и не показывает окон. Модель только хранит и изменяет данные и отвечает значениями. Все сообщения пользователю будут добавлены позже, в интерфейс. Это и есть разделение ответственности.

Шаг 4 app.py: импорты. Первые три строки приносят графику: модуль tkinter под коротким именем, улучшенные виджеты ttk и диалоги messagebox. Четвёртая строка подключает модель: после неё в app.py можно вызывать find_user, add_user и остальные так, будто они объявлены здесь.

```
import os
import random
import tkinter as tk
from tkinter import ttk, messagebox

from models import (all_users, find_user, login_exists, add_user,
                    unlock_user, register_fail, reset_attempts)
```

Это и есть «подключение модели к интерфейсу». Интерфейс не знает, список внутри модели или база данных: он знает только имена функций.

Шаг 5 app.py: окно, контейнер и две общие переменные. Создаётся главное окно, ему задаются заголовок, размер и цвет фона. Внутри окна создаётся контейнер Frame, растянутый на всё окно; параметры padx=20, pady=20 у его pack дают внешний отступ в двадцать пикселей со всех сторон, то есть поля вокруг содержимого каждого экрана. Переменная CURRENT_USER на верхнем уровне хранит словарь вошедшего пользователя и пока пуста. Константа BG задаёт общий цвет один раз, и все виджеты ссылаются на неё, поэтому смена цвета схемы делается правкой одной строки. Рядом вводятся две константы для полей ввода: FIELD_BG — белый фон поля и FIELD_FG — цвет текста и курсора в нём. Поля окрашиваются явно и получают видимую рамку, чтобы оставаться заметными на любой теме оформления операционной системы, даже если платформа проигнорирует цвет окна. Все цвета и шрифты собраны в константы в начале файла, и больше нигде в коде hex-коды и кортежи шрифтов не встречаются. Перекраска всего интерфейса выполняется правкой констант в одном месте, без поиска цветов по экранам. Введены два готовых стиля кнопок: основная (PRIMARY_BG, синяя) для главного действия экрана и второстепенная (SECOND_BG, светло-серая) для всех остальных. Поля ввода всегда белые с видимой рамкой (FIELD_BG, FIELD_FG), поэтому читаются на любом фоне.

```

CURRENT_USER = None

BG = "#f9f1e5"
FIELD_BG = "#ffffff"
FIELD_FG = "#22262b"
PRIMARY_BG = "#1e5fd0"
PRIMARY_FG = "#ffffff"
SECOND_BG = "#e5e7eb"
SECOND_FG = "#111827"

FONT_TITLE = ("Arial", 16, "bold")
FONT_LABEL = ("Arial", 11, "bold")
FONT_BUTTON = ("Arial", 12, "bold")

root = tk.Tk()
root.title("Учебное приложение")
root.geometry("640x480")
root.configure(bg=BG)

container = tk.Frame(root, bg=BG)
container.pack(fill="both", expand=True, padx=20, pady=20)

```

Шаг 6 app.py: очистка экрана и строители виджетов. Первая функция получает список всех виджетов, лежащих непосредственно в контейнере, и уничтожает каждый: контейнер становится пустым, предыдущий экран исчезает. Остальные четыре функции — строители виджетов. Каждая создаёт и **возвращает** готовый виджет, уже окрашенный и со шрифтом, но не размещает его: размещение выполнит вызывающий код своим `.pack(...)`, потому что у разных экранов разные отступы. Благодаря строителям весь код оформления собран в одном месте, а экраны состоят только из понятных строк «создать надпись», «создать поле», «создать кнопку». У `make_entry` параметр `password` переключает поле в режим пароля. У `make_button` параметр `primary` выбирает синюю основную кнопку или серую второстепенную, а параметр `parent` позволяет положить кнопку не в контейнер, а в другую рамку, что понадобится для ряда кнопок под таблицей.

```

def clear_screen():
    for w in container.winfo_children():
        w.destroy()

def make_title(text):
    return tk.Label(container, text=text, bg=BG, fg=FIELD_FG, font=FONT_TITLE)

def make_label(text):
    return tk.Label(container, text=text, bg=BG, fg=FIELD_FG,
                    font=FONT_LABEL, anchor="w")

def make_entry(password=False):
    entry = tk.Entry(container, font=FONT_BUTTON, bg=FIELD_BG, fg=FIELD_FG,
                    insertbackground=FIELD_FG, relief="solid", borderwidth=1)
    if password:
        entry.configure(show="*")
    return entry

def make_button(text, command, primary=True, parent=None):
    host = container if parent is None else parent
    if primary:
        return tk.Button(host, text=text, command=command,
                        bg=PRIMARY_BG, fg=PRIMARY_FG,
                        activebackground="#174ea6", activeforeground=PRIMARY_FG,
                        font=FONT_BUTTON, relief="flat", cursor="hand2")
    return tk.Button(host, text=text, command=command,
                    bg=SECOND_BG, fg=SECOND_FG,
                    activebackground="#d7dade", activeforeground=SECOND_FG,
                    font=FONT_LABEL, relief="flat", cursor="hand2")

```

Шаг 7 app.py: интерактивная капча-пазл из фрагментов картинки. Положите рядом с app.py папку captcha и в неё четыре файла piece_1.png, piece_2.png, piece_3.png, piece_4.png — это четыре фрагмента одной картинки. Константа CORRECT_ORDER задаёт правильный порядок раскладки фрагментов по клеткам поля два на два. Функция load_captcha_images читает PNG средствами самого tkinter через tk.PhotoImage(file=...) и уменьшает каждый фрагмент вызовом subsample(6); сторонняя библиотека для изображений не нужна. Словарь captcha_images хранит ссылки на изображения, иначе сборщик мусора их удалит и кнопки станут пустыми.

```

BASE_DIR = os.path.dirname(os.path.abspath(__file__))
CAPTCHA_DIR = os.path.join(BASE_DIR, "captcha")
CORRECT_ORDER = [1, 2, 4, 3]
PIECE_SIZE = 110

captcha_images = {}
captcha_host = None
captcha_slots = []
captcha_piece_buttons = {}
captcha_placed = []
captcha_shuffled = []
captcha_passed = False

def load_captcha_images():
    global captcha_images
    captcha_images = {}
    for piece in (1, 2, 3, 4):
        path = os.path.join(CAPTCHA_DIR, "piece_" + str(piece) + ".png")
        captcha_images[piece] = tk.PhotoImage(file=path).subsample(6)

def build_captcha(host):
    global captcha_slots, captcha_piece_buttons, captcha_placed, captcha_shuffled
    for w in host.winfo_children():
        w.destroy()
    captcha_slots = []
    captcha_piece_buttons = {}
    captcha_placed = []
    captcha_shuffled = CORRECT_ORDER[:]
    random.shuffle(captcha_shuffled)
    while captcha_shuffled == CORRECT_ORDER:
        random.shuffle(captcha_shuffled)
    tk.Label(host, text="Соберите картинку: кликайте фрагменты по порядку",
             bg=BG, fg=FIELD_FG, font=FONT_LABEL, anchor="w").pack(fill="x")
    board = tk.Frame(host, bg=BG)
    board.pack(pady=(6, 6))
    for index in range(4):
        slot = tk.Label(board, width=PIECE_SIZE, height=PIECE_SIZE,
                        bg=FIELD_BG, relief="solid", borderwidth=1)
        slot.grid(row=index // 2, column=index % 2)
        captcha_slots.append(slot)
    pieces = tk.Frame(host, bg=BG)
    pieces.pack(pady=(0, 6))
    for piece in captcha_shuffled:
        btn = tk.Button(pieces, image=captcha_images[piece],
                        relief="solid", borderwidth=1, cursor="hand2",
                        command=lambda p=piece: place_piece(p))
        btn.pack(side="left", padx=3)
        captcha_piece_buttons[piece] = btn
    make_button("Сбросить капчу", new_captcha_round, primary=False,

```

```

parent=host).pack(ipadx=16, ipady=3)

def new_captcha_round():
    global captcha_passed
    captcha_passed = False
    build_captcha(captcha_host)

def place_piece(piece):
    if captcha_passed or len(captcha_placed) >= 4:
        return
    captcha_placed.append(piece)
    captcha_piece_buttons[piece].configure(state="disabled")
    captcha_slots[len(captcha_placed) - 1].configure(image=captcha_images[piece])
    if len(captcha_placed) == 4:
        check_captcha()

def check_captcha():
    global captcha_passed
    login = ENTRY_LOGIN.get().strip()
    if captcha_placed == CORRECT_ORDER:
        captcha_passed = True
        messagebox.showinfo("Капча",
                             "Пазл собран верно. Теперь можно нажимать «Войти».")
        return
    became_locked = register_fail(login)
    if became_locked:
        messagebox.showerror("Блокировка",
                             "Вы заблокированы. Обратитесь к администратору")
    else:
        messagebox.showerror("Капча",
                             "Пазл собран неверно. Попробуйте собрать ещё раз, "
                             "ориентируясь на продолжение рисунка.")
    new_captcha_round()

```

Разбор по частям. `build_captcha` сначала удаляет всё содержимое своей рамки, чтобы пересобрать капчу заново, затем перемешивает порядок кнопок-фрагментов и гарантирует, что перемешанный порядок не совпал случайно с правильным. Поле сборки — это рамка с четырьмя пустыми клетками-надписями, разложенными менеджером `grid` по схеме два на два; внутри этой рамки используется именно `grid`, хотя весь остальной экран собран на `pack`, потому что это другой контейнер и конфликтов нет. Кнопки-фрагменты несут изображение через параметр `image` и при клике вызывают `place_piece` со своим номером через `lambda p=piece`. `place_piece` добавляет номер в список собранных, отключает нажатую кнопку, чтобы фрагмент нельзя было поставить дважды, и кладёт изображение в следующую свободную клетку. Когда собрано четыре фрагмента, вызывается `check_captcha`: она сравнивает полученный список с `CORRECT_ORDER`; при совпадении ставит флаг `captcha_passed`, при несовпадении засчитывает неудачную попытку через `register_fail` и пересобирает капчу.

Шаг 8 app.py: проверка входа, счётчик попыток и блокировка. Функция `try_login` разбирается построчно. Сначала объявляется `global CURRENT_USER`, иначе присваивание создало бы локальную переменную и следующий экран не узнал бы, кто вошёл. Затем ввод очищается от пробелов и проверяется на пустоту. Далее стоит гейт капчи: пока флаг `captcha_passed` не поднят, вход запрещён предупреждением. Вызов `find_user` отдаёт словарь пользователя или `None`; при `None` вызывается `register_fail`, который увеличивает счётчик неудачных попыток этого логина и при третьем неудачном подряд устанавливает блокировку, возвращая `True`. Поэтому текст сообщения выбирается по возвращённому флагу: либо о блокировке, либо дословное сообщение технического задания о неверной паре. Отдельно проверяется флаг блокировки уже найденного пользователя: заблокированного не пускают даже с верным паролем, и попытка не считается. Только после всех отказов счётчик сбрасывается вызовом `reset_attempts`, пользователь запоминается в `CURRENT_USER`, показывается сообщение об успешной авторизации, капча пересобирается для следующего входа и выполняется переход по роли: администратор попадает на панель администратора, остальные на рабочее место пользователя.

```

def try_login(login, password):
    global CURRENT_USER
    login = login.strip()
    if not login or not password:
        messagebox.showwarning("Внимание", "Заполните логин и пароль.")
        return
    if not captcha_passed:
        messagebox.showwarning("Капча",
                                "Сначала соберите капчу, затем нажимайте «Войти».")
        return
    user = find_user(login, password)
    if user is None:
        became_locked = register_fail(login)
        if became_locked:
            messagebox.showerror("Блокировка",
                                "Вы заблокированы. Обратитесь к администратору")
        else:
            messagebox.showerror(
                "Ошибка входа",
                "Вы ввели неверный логин или пароль. "
                "Пожалуйста проверьте ещё раз введенные данные")
            return
    if user["locked"]:
        messagebox.showerror("Блокировка",
                                "Вы заблокированы. Обратитесь к администратору")
        return
    reset_attempts(login)
    CURRENT_USER = user
    messagebox.showinfo("Авторизация", "Вы успешно авторизовались")
    new_captcha_round()
    if user["role"] == "Администратор":
        open_admin()
    else:
        open_user()

```

Функция `open_login` строит экран входа. Она начинается с очистки контейнера и заголовка. Первым элементом создаётся рамка `captcha_host` под капчу и в неё вызовом `build_captcha` помещается пазл; капча стоит выше полей, чтобы пользователь сначала собрал её. Переменные `ENTRY_LOGIN` и `captcha_host` объявлены через `global`, потому что к полю логина обращается проверка капчи, а рамка капчи пересоздаётся из других функций. Дальше идут две пары «надпись плюс поле», причём поле пароля скрывает ввод звёздочками. Кнопка «Войти» через `lambda` читает оба поля в момент нажатия и передаёт строки в `try_login`. Вторая кнопка, серая, ведёт на экран «О программе» и передаёт функцию напрямую, без аргументов, поэтому `lambda` ей не нужна.

```

def open_login():
    global ENTRY_LOGIN, captcha_host
    clear_screen()
    make_title("Вход в систему").pack(pady=(0, 10))

    captcha_host = tk.Frame(container, bg=BG)
    captcha_host.pack(fill="x", pady=(0, 10))
    build_captcha(captcha_host)

    make_label("Логин").pack(fill="x")
    ENTRY_LOGIN = make_entry()
    ENTRY_LOGIN.pack(fill="x", pady=(0, 8))
    make_label("Пароль").pack(fill="x")
    entry_pass = make_entry(password=True)
    entry_pass.pack(fill="x", pady=(0, 12))

    make_button("Войти",
                lambda: try_login(ENTRY_LOGIN.get(), entry_pass.get())
                ).pack(fill="x", ipady=6)
    make_button("0 программе", open_about, primary=False
                ).pack(fill="x", pady=(8, 0), ipady=4)

```

Шаг 9 app.py: разблокировка и панель администратора. Функция `unlock_selected` принимает таблицу как параметр. Сначала она смотрит, есть ли выделенная строка; если выделения нет, показывает предупреждение и завершается, не пытаясь читать несуществующую строку. Затем из выделенной строки берётся значение первой колонки, то есть логин. Вызов `unlock_user` из модели меняет флаг блокировки, после чего показывается подтверждение и экран админа открывается заново, чтобы таблица перерисовалась с новыми данными.

```

def unlock_selected(tree):
    selected = tree.selection()
    if not selected:
        messagebox.showwarning("Внимание", "Сначала выберите строку в таблице.")
        return
    login = tree.item(selected[0])["values"][0]
    unlock_user(login)
    messagebox.showinfo("Готово", "Пользователь " + str(login) + " разблокирован.")
    open_admin()

```

Функция `open_admin` строит панель. После очистки и заголовка ставится строка приветствия, в которую подставляется логин из `CURRENT_USER`; если бы вход не произошёл, этой переменной не было бы значения, поэтому админ-панель и открывается только из успешного входа. Затем создаётся таблица: перечисляются внутренние имена колонок, для каждой задаётся видимый заголовок и ширина с выравниванием. Цикл проходит по `all_users()` и вставляет строку за строкой; логический флаг блокировки превращается в понятные слова «да» и «нет» прямо в момент вставки. Таблица растягивается на всё свободное место. Ниже создаётся невидимая рамка-ряд, в которую кнопки складываются слева направо через `side="left"` с зазором шесть пикселей; кнопка «Выйти» прижата к правому краю через `side="right"`. Каждая кнопка своего

цвета по смыслу действия: добавить — зелёная, разблокировать — янтарная, оформление — фиолетовая, выйти — красная.

```
def open_admin():
    clear_screen()
    make_title("Панель администратора").pack(pady=(0, 5))
    make_label("Вы вошли как: " + CURRENT_USER["login"]).pack(fill="x", pady=(0, 10))

    tree = ttk.Treeview(container, columns=("login", "role", "locked"),
                        show="headings", height=8)
    tree.heading("login", text="Логин")
    tree.heading("role", text="Роль")
    tree.heading("locked", text="Заблокирован")
    tree.column("login", width=180, anchor="w")
    tree.column("role", width=160, anchor="w")
    tree.column("locked", width=120, anchor="center")
    for u in all_users():
        tree.insert("", "end",
                    values=(u["login"], u["role"],
                            "да" if u["locked"] else "нет"))
    tree.pack(fill="both", expand=True, pady=(0, 10))

    row = tk.Frame(container, bg=BG)
    row.pack(fill="x")
    make_button("Добавить", open_add_user, parent=row).pack(side="left", padx=(0, 6))
    make_button("Разблокировать", lambda: unlock_selected(tree),
                primary=False, parent=row).pack(side="left", padx=(0, 6))
    make_button("Оформление", open_style,
                primary=False, parent=row).pack(side="left", padx=(0, 6))
    make_button("Выйти", open_login,
                primary=False, parent=row).pack(side="right")
```

Кнопка «Разблокировать» использует lambda, потому что передаёт в функцию конкретную таблицу tree, созданную на этом экране.

Шаг 10 ipr.ru: рабочее место пользователя. Экран для роли «Пользователь»: заголовок, две строки с логином и ролью из CURRENT_USER и кнопка возврата на вход. Он нужен, чтобы успешный вход неадминистратора не приводил на панель администратора, и чтобы у каждой роли был свой экран с кнопкой выхода.

```
def open_user():
    clear_screen()
    make_title("Рабочее место пользователя").pack(pady=(0, 10))
    make_label("Вы вошли как: " + CURRENT_USER["login"]).pack(fill="x", pady=(0, 6))
    make_label("Роль: " + CURRENT_USER["role"]).pack(fill="x", pady=(0, 20))
    make_button("Выйти", open_login, primary=False).pack(ipadx=24, ipady=4)
```

Шаг 11 ipr.ru: сохранение нового пользователя и форма добавления. Функция save_user выполняет три проверки подряд, и каждая завершается возвратом: пустой логин или пароль

отклоняются предупреждением; занятый логин отклоняется ошибкой, для чего вызывается `login_exists` из модели; только затем вызывается `add_user`, показывается подтверждение и открывается панель администратора, где новый пользователь сразу виден в таблице.

```
def save_user(login, password, role):
    login = login.strip()
    if not login or not password:
        messagebox.showwarning("Внимание", "Логин и пароль не могут быть пустыми.")
        return
    if login_exists(login):
        messagebox.showerror("Ошибка", "Такой логин уже существует.")
        return
    add_user(login, password, role)
    messagebox.showinfo("Готово", "Пользователь " + login + " добавлен.")
    open_admin()
```

Функция `open_add_user` строит форму из двух пар «надпись плюс поле» и выпадающего списка ролей. Список создан со `state="readonly"`, чтобы роль нельзя было вписать руками, и вызов `current(0)` заранее выбирает первый вариант, иначе поле стояло бы пустым. Кнопка «Сохранить» через `lambda` передаёт в `save_user` три значения: логин, пароль и выбранную роль. Кнопка «Отмена» просто возвращает на панель администратора, ничего не сохраняя.

```
def open_add_user():
    clear_screen()
    make_title("Новый пользователь").pack(pady=(0, 15))
    make_label("Логин").pack(fill="x")
    e_login = make_entry()
    e_login.pack(fill="x", pady=(0, 10))
    make_label("Пароль").pack(fill="x")
    e_pass = make_entry()
    e_pass.pack(fill="x", pady=(0, 10))
    make_label("Роль").pack(fill="x")
    combo_role = ttk.Combobox(container, values=["Пользователь", "Администратор"],
                              state="readonly", font=FONT_BUTTON)

    combo_role.current(0)
    combo_role.pack(fill="x", pady=(0, 15))
    make_button("Сохранить",
                lambda: save_user(e_login.get(), e_pass.get(), combo_role.get())
                ).pack(fill="x", ipady=6)
    make_button("Отмена", open_admin, primary=False).pack(fill="x", pady=(8, 0), ipac
```

Шаг 12 **ipr.ru: экран «О программе»**. Самый простой экран: три надписи разного размера, цвета и насыщенности, показывающие, как параметры `font` и `fg` меняют вид текста, и кнопка возврата на вход. Отступы сверху у первой надписи сделаны кортежем, чтобы опустить блок к центру окна.

```
def open_about():
    clear_screen()
    tk.Label(container, text="Учебное приложение", bg=BG, fg=PRIMARY_BG,
              font=("Arial", 20, "bold")).pack(pady=(30, 6))
    tk.Label(container, text="Модуль 4. Демонстрационный экзамен 09.02.07",
              bg=BG, fg=FIELD_FG, font=FONT_BUTTON).pack(pady=(0, 4))
    tk.Label(container, text="Версия 1.0, данные пока хранятся в списке",
              bg=BG, fg=FIELD_FG, font=FONT_LABEL).pack(pady=(0, 25))
    make_button("Назад", open_login, primary=False).pack(ipadx=30, ipady=5)
```

Шаг 13 ipr.py: экран «Оформление». Экран демонстрирует цвета и lambda с аргументом.

Пояснительная надпись имеет wraplength, чтобы длинный текст переносился по ширине, а не уходил за окно. Ниже создаётся плашка: рамка фиксированной высоты сто двадцать пикселей. Вызов pack_propagate(False) запрещает рамке сжиматься под содержимое; без него пустая рамка схлопнулась бы в ноль пикселей и её не было бы видно. Три кнопки перекрашивают плашку, каждая в свой цвет; действие одно и то же, preview.configure(bg=...), но цвет у каждой кнопки свой, поэтому и нужна lambda с аргументом. Кнопка «Назад» возвращает на панель администратора.

```
def open_style():
    clear_screen()
    make_title("Оформление").pack(pady=(0, 15))
    make_label("Кнопки меняют цвет плашки ниже.").pack(fill="x", pady=(0, 12))

    preview = tk.Frame(container, bg=FIELD_BG, height=120)
    preview.pack(fill="x", pady=(0, 15))
    preview.pack_propagate(False)

    make_button("Красный", lambda: preview.configure(bg="#ffd6d6"),
                primary=False).pack(fill="x", ipady=4, pady=(0, 6))
    make_button("Зелёный", lambda: preview.configure(bg="#d6f5e0"),
                primary=False).pack(fill="x", ipady=4, pady=(0, 6))
    make_button("Синий", lambda: preview.configure(bg="#d6e4ff"),
                primary=False).pack(fill="x", ipady=4, pady=(0, 6))
    make_button("Назад", open_admin, primary=False).pack(ipadx=30, ipady=5, pady=(10,
```

Шаг 14 ipr.py: старт приложения. В самый конец файла добавляются две строки. Первая открывает стартовый экран, вторая запускает цикл событий; без неё окно не появится и программа сразу завершится.

```
load_captcha_images()
open_login()
root.mainloop()
```

Запустите app.py и пройдите весь маршрут: вход под парой admin / admin; просмотр таблицы; разблокировка ivanov и проверка, что после перерисовки в таблице стоит «нет»; добавление нового пользователя и его появление в таблице; выход; попытка входа под ivanov до

разблокировки, если вы перезапустили программу, с ожиданием сообщения о блокировке; экран «О программе» и возврат; экран «Оформление» и перекраска плашки тремя кнопками.

Проверьте себя после сборки

- Вход с парой `admin / admin` открывает панель администратора.
- Вход с неверным паролем показывает окно ошибки и остаётся на экране входа.
- Вход под заблокированным `ivanov` показывает «Учётная запись заблокирована».
- Пустой логин или пароль показывает предупреждение, а не падает.
- Добавление пользователя с уже существующим логином отклоняется.
- После добавления новый пользователь виден в таблице.
- Кнопка «Разблокировать» без выделенной строки показывает предупреждение, а не падает.
- В `models.py` нет ни одной строки с `tkinter`.
- Нажатие «Войти» до сборки капчи показывает предупреждение и не выполняет вход.
- Неверно собранный пазл показывает ошибку, засчитывает попытку и пересобирается в новом порядке.
- Три неудачи подряд, паролем или пазлом, блокируют учётную запись; повторный вход даёт сообщение «Вы заблокированы. Обратитесь к администратору».
- После разблокировки через панель администратора счётчик попыток обнулён и вход снова возможен.
- Вход под ролью «Пользователь» открывает рабочее место пользователя, а не панель администратора.
- Ни одно действие не приводит к аварийному завершению программы.

4. Задания для самостоятельной работы

Выполняются по порядку, каждое опирается на предыдущее. Сдаются изменённые `models.py` и `app.py`. Задания сформулированы в рамках требований модуля 4: раскладка, цвета, шрифты, диалоги, таблицы и проверки ввода.

№	Задание	Что отрабатывается
1	Поменять цветовую схему, правя только константы в начале файла: <code>BG</code> , <code>PRIMARY_BG</code> , <code>SECOND_BG</code> . Подобрать собственные светлые пастельные оттенки и убедиться, что весь интерфейс перекрасился без правки экранов.	Константы цветов, понимание выноса оформления в одно место
2	Изменить размеры и семейства шрифтов, правя только константы <code>FONT_TITLE</code> , <code>FONT_LABEL</code> , <code>FONT_BUTTON</code> ; добиться видимой разницы между заголовком, подписями и кнопками.	Константы шрифтов, параметр <code>font</code>
3	Найти самостоятельно и применить курсивное и полужирно-курсивное начертание шрифта: оформить пояснительный текст экрана «О программе» курсивом.	Самостоятельный поиск значений <code>font</code>
4	Добавить на экран входа флажок «Показать пароль», переключающий	<code>tk.Checkbutton</code> , <code>.configure(show=...)</code>

	параметр show поля пароля между "*" и пустой строкой.	
5	Добавить над капчей надпись с числом оставшихся попыток для введённого логина, обновляемую после каждой неудачи и обнуляемую при успехе.	Поле attempts модели, f-строки, обновление надписи
6	Реализовать альтернативный способ сборки капчи перетаскиванием фрагментов мышью вместо кликов, сохранив ту же проверку порядка.	События кнопки, bind, перетаскивание
7	Добавить в таблицу админ-панели четвёртую колонку и кнопку «Удалить», убирающую выделенного пользователя через новую функцию модели; пользователя admin удалять запретить.	Treeview, selection, item, новая функция модели
8	Сделать экран «Изменить пользователя»: по выделенной строке открывать форму с предзаполненными полями и кнопкой «Сохранить изменения».	Передача данных между экранами, .insert(0, ...)
9	Добавить над таблицей поле поиска и кнопку «Найти», оставляющую в таблице только пользователей, чей логин содержит введённый фрагмент.	Фильтрация списка, перерисовка таблицы
10	Перевести раскладку экрана входа с pack на grid: надписи в левой колонке, поля в правой, кнопка на две колонки; настроить растягивание колонки с полями.	Сетка grid, sticky, colspan, rowspan
11	Добавить внизу окна строку состояния: надпись с логином вошедшего и названием текущего экрана, обновляемую при каждом переходе.	Отдельный виджет вне контейнера, обновление текста
12	Запретить изменение размера окна и вывести его по центру экрана при старте, самостоятельно найдя формулу координат через размеры экрана.	resizable, wininfo_screenwidth, geometry с координатами
13	Перед выходом с панели администратора показывать диалог подтверждения askyesno и выходить только при ответе «Да».	Модальный диалог с возвращаемым значением

14	Оформить таблицу: задать чередование цвета строк через теги Treeview и изменить вид заголовков колонок через ttk.Style, самостоятельно найдя названия параметров стиля.	Теги Treeview, ttk.Style, самостоятельный поиск
15	Добавить кнопку «Обновить» и надпись над таблицей с общим числом пользователей; оба значения пересчитывать при каждом открытии экрана.	Перерисовка экрана, обращение к модели за количеством

5. Типичные ошибки и как их читать

Traceback читается снизу вверх: последняя строка — тип и сообщение ошибки, строки выше — путь к месту. Ниже частые для этой практики случаи.

Сообщение или симптом	Причина	Как исправить
ModuleNotFoundError: No module named 'models'	Файл models.py лежит не в той папке, что app.py, или назван иначе	Положить оба файла в одну папку проекта и открыть эту папку в редакторе
NameError: name 'open_admin' is not defined	Функция вызывается раньше, чем определена в файле	Переместить определение выше места вызова либо вызывать только из обработчиков, срабатывающих после загрузки файла
TypeError: 'NoneType' object is not subscriptable на строке с CURRENT_USER["login"]	Экран админа открыт до успешного входа, CURRENT_USER ещё None	Открывать админ-панель только после успешной проверки логина
IndentationError: expected an indented block	После двоеточия нет строки с отступом	Добавить тело блока с отступом в 4 пробела
IndentationError: unindent does not match...	Смешаны пробелы и табуляция	В редакторе: Ctrl+Shift+P → «Convert Indentation to Spaces»
Кнопка срабатывает сразу при запуске, а не по нажатию	Написано command=func() со скобками	Убрать скобки; если нужны аргументы — command=lambda: func(...)
tkinter.TclError о конфликте менеджеров размещения	В одном контейнере смешаны pack и grid	Внутри одного контейнера использовать только один

		менеджер; для разных раскладок создать вложенные Frame
Окно мигнуло и закрылось	Не вызван <code>root.mainloop()</code>	Добавить <code>root.mainloop()</code> последней строкой
Таблица Treeview пустая, хотя данные есть	Не выполнен цикл вставки или перепутан порядок значений в <code>values</code>	Проверить цикл по <code>all_users()</code> и порядок колонок
Плашка экрана «Оформление» не видна	Пустой Frame сжался до нулевой высоты	Задать <code>height</code> и вызвать <code>pack_propagate(False)</code>
После добавления или разблокировки таблица не обновилась	Экран не перерисован	После изменения данных вызвать функцию экрана заново
<code>ValueError: invalid literal for int()</code>	В поле ввода буквы, а код делает <code>int(entry.get())</code>	Обернуть преобразование в <code>try / except ValueError</code> и показать предупреждение
Изменение данных в интерфейсе не видно после перезапуска	Модель хранит список в памяти и не сохраняет его между запусками	Это ожидаемо на текущем шаге; постоянное хранение появится вместе с базой данных
<code>ModuleNotFoundError: No module named 'tkinter'</code> или обрыв трассировки на строке <code>import _tkinter</code>	В системе установлен Python без компонента tkinter	Windows: доустановить компонент «tcl/tk and Idle» через Modify установщика. Linux: поставить системный пакет <code>python3-tk</code> (Fedora — <code>python3-tkinter</code> , ALT — <code>python3-module-tkinter</code>). Порядок действий в разделе 2
Окно открывается серым или тёмным вместо заданного светлого фона	Цвет фона не задан у окна и контейнера либо задан только у части виджетов	Задать <code>root.configure(bg=BG)</code> , <code>bg=BG</code> у контейнера и у каждой надписи; поля ввода держать белыми с рамкой

6. Контрольные вопросы

1. Зачем нужна строка `root.mainloop()` и что произойдёт без неё?
2. Чем отличаются параметры `bg` и `fg`? Где задаётся цвет всего окна?
3. Чем внешний отступ отличается от внутреннего и какими параметрами задаётся каждый?
4. Приведите ситуацию, где внутренний отступ предпочтительнее увеличения шрифта.
5. Что означают записи `padx=(0, 10)` и `padx=(0, 6)`?
6. Что такое сетка `grid` и какие у неё основные аргументы? Какой менеджер размещения проще всего и почему?
7. Почему в одном контейнере нельзя смешивать `pack` и `grid` и что делать, если нужны обе раскладки?
8. Почему виджет, созданный без вызова менеджера размещения, не виден на экране?
9. Что принимает параметр `command` у кнопки и почему без скобок? Когда необходима `lambda`?
10. Что такое модальное диалоговое окно и чем `showerror` отличается от `askyesno` по возвращаемому значению?

11. Что возвращает `Entry.get()` и почему результат нужно проверять и преобразовывать?
12. Зачем проект разбит на `models.py` и `app.py`? Что изменится при переходе с списка на базу данных?
13. Почему все функции экранов оставлены в одном файле, а модель вынесена отдельно?
14. Как устроена смена экранов и зачем нужна функция `clear_screen()`?
15. Зачем в `try_login` стоит строка `global CURRENT_USER`?
16. Что делает `show="headings"` у `Treeview` и что случится без него?
17. Почему после изменения данных функция экрана вызывается повторно?
18. Какие проверки выполняет `save_user` перед добавлением записи и зачем каждая?
19. Где в приложении обрабатываются ситуации «пустое поле» и «выделенной строки нет», и почему программа при них не падает?